EXPERIMENT NUMBER :1.a

Analyze the trend of data science job postings over the last decade for the given data on the number of data science postings each year . Use pandas for data manipulation and matplotlib /seaborn for visualization.

CODE:

```
import pandas as pd
import matplotlib.pyplot as plt

# Create data
year = [2015, 2016, 2017, 2018, 2019,
      2020, 2021, 2022, 2023, 2024]

job_postings = [1200, 1500, 1800, 2300, 3000,
            2700, 4000, 5200, 6500, 7800]

# Create DataFrame
df = pd.DataFrame({
   'Year': year,
   'Job Postings': job_postings
})

# Display the data
print(df)

# Plot the graph
plt.plot(df['Year'], df['Job Postings'], marker='o')

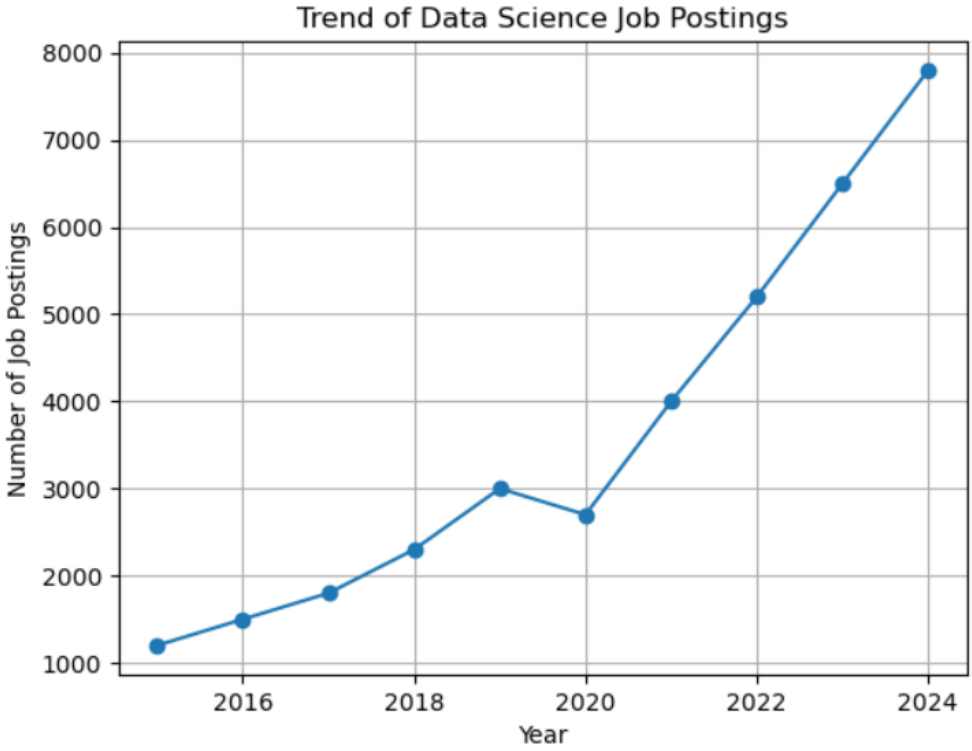
plt.title('Trend of Data Science Job Postings')
plt.xlabel('Year')
plt.ylabel('Number of Job Postings')
plt.grid()

plt.show()
```

OUTPUT:

```
   Year      Job Postings
0  2015        1200
1  2016        1500
2  2017        1800
```

3	2018	2300
4	2019	3000
5	2020	2700
6	2021	4000
7	2022	5200
8	2023	6500
9	2024	7800



EXPERIMENT NUMBER :1.b

Analyze and visualize the distribution of various data science roles (Data Analyst, Data Engineer, Data Scientist, etc.) from a dataset.

CODE:

```
import pandas as pd
import matplotlib.pyplot as plt

# Create a simple dataset
data = {
   "Job Role": [
      "Data Analyst",
      "Data Scientist",
      "Data Engineer",
      "Data Analyst",
```

```
        "Data Scientist",
        "Data Engineer",
        "Data Analyst",
        "Data Scientist",
        "Data Analyst",
        "Data Engineer",
        "Data Scientist",
        "Data Analyst"
    ]
}

# Convert the data into a DataFrame
df = pd.DataFrame(data)

# Display the dataset
print("Dataset:")
print(df)

# Count the number of people in each job role
role_count = df["Job Role"].value_counts()

# Display the distribution
print("\nDistribution of Data Science Roles:")
print(role_count)

# Create a bar chart
plt.figure(figsize=(8, 5))

plt.bar(role_count.index, role_count.values)

# Add title and labels
plt.title("Distribution of Data Science Roles")
plt.xlabel("Job Role")
plt.ylabel("Number of People")

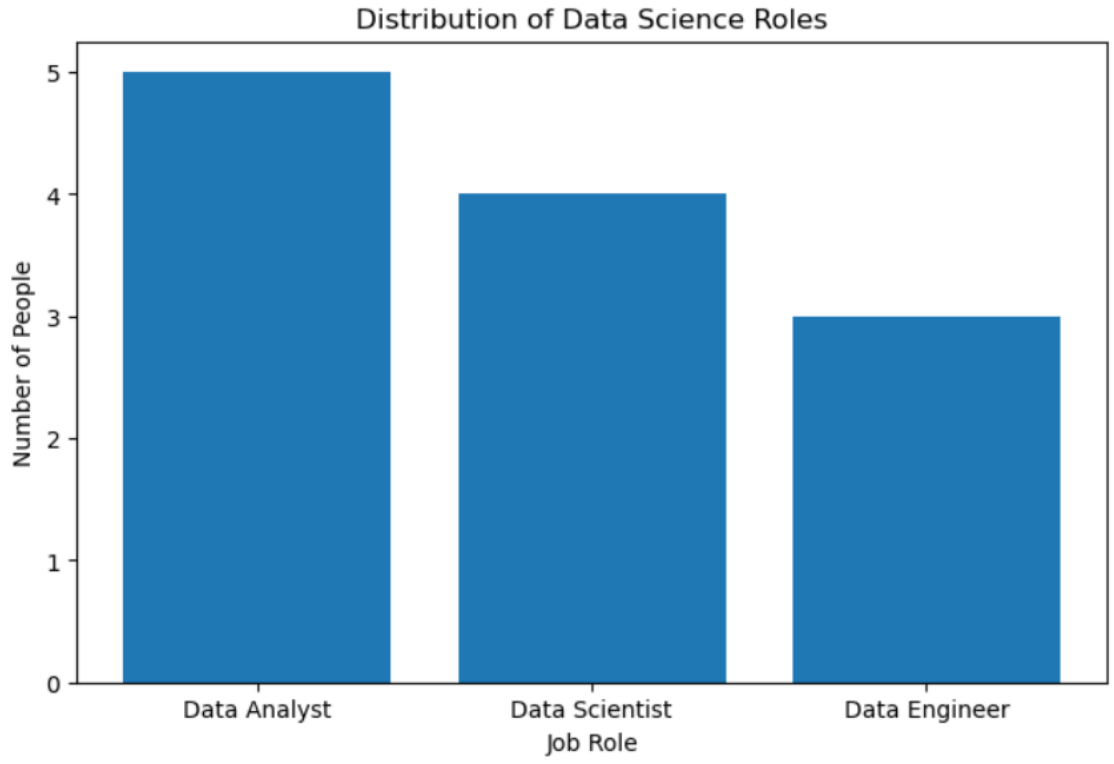
# Display the graph
plt.show()
```

OUTPUT :

```
Dataset:
        Job Role
```

0 Data Analyst
1 Data Scientist
2 Data Engineer
3 Data Analyst
4 Data Scientist
5 Data Engineer
6 Data Analyst
7 Data Scientist
8 Data Analyst
9 Data Engineer
10 Data Scientist
11 Data Analyst

Distribution of Data Science Roles:
Job Role
Data Analyst 5
Data Scientist 4
Data Engineer 3
Name: count, dtype: int64



EXPERIMENT NUMBER :1.c

Conduct an experiment to differentiate Structured, Un-structured and Semi structured data based on data sets given.

CODE:

```
# Import pandas
import pandas as pd

# ------------------------------
# 1. STRUCTURED DATA
# ------------------------------

# Structured data is arranged in rows and columns
structured_data = {
  "Name": ["Arun", "Priya", "Rahul"],
  "Age": [20, 21, 19],
  "Department": ["CSE", "ECE", "CSE"]
}

structured = pd.DataFrame(structured_data)

print("STRUCTURED DATA")
print(structured)



# ------------------------------
# 2. SEMI-STRUCTURED DATA
# ------------------------------

# Semi-structured data has some organization
# but does not follow a fixed table format

semi_structured = {
  "student1": {
    "Name": "Arun",
    "Age": 20,
    "Department": "CSE"
  },
  "student2": {
    "Name": "Priya",
    "Age": 21,
    "Department": "ECE"
  }
}

print("\nSEMI-STRUCTURED DATA")
```

```
print(semi_structured)



# ------------------------------
# 3. UNSTRUCTURED DATA
# ------------------------------

# Unstructured data does not have a fixed format

unstructured = [
   "Arun is a CSE student and he is 20 years old.",
   "Priya is studying ECE and she likes photography.",
   "The students attended a technical event yesterday."
]

print("\nUNSTRUCTURED DATA")

for text in unstructured:
   print(text)
```

OUTPUT:

```
STRUCTURED DATA
   Name  Age Department
0  Arun   20       CSE
1  Priya  21       ECE
2  Rahul  19       CSE

SEMI-STRUCTURED DATA
{'student1': {'Name': 'Arun', 'Age': 20, 'Department': 'CSE'}, 'student2': {'Name': 'Priya',
'Age': 21, 'Department': 'ECE'}}

UNSTRUCTURED DATA
Arun is a CSE student and he is 20 years old.
Priya is studying ECE and she likes photography.
The students attended a technical event yesterday.
```

EXPERIMENT NUMBER :1.d

Conduct an experiment to encrypt and decrypt given sensitive data Use the cryptography library fernet.

CODE:

```
from cryptography.fernet import Fernet

# Generate a secret key
key = Fernet.generate_key()

print("Secret Key:")
print(key)

# Create Fernet object
cipher = Fernet(key)

# Given sensitive data
data = "My password is Python123"

print("\nOriginal Data:")
print(data)

# Encrypt the data
encrypted_data = cipher.encrypt(data.encode())

print("\nEncrypted Data:")
print(encrypted_data)

# Decrypt the data
decrypted_data = cipher.decrypt(encrypted_data).decode()

print("\nDecrypted Data:")
print(decrypted_data)
```

OUTPUT:

```
Secret Key:
b'Z3tqj0PnCc9lVNX8nMo7yDmknTZCUoXyScxNwkU004E='

Original Data:
My password is Python123

Encrypted Data:
b'gAAAAABqx1qpCfy1gep0_ckTXbZjV0kxcQIAn7IPhxu7kGm0aeJ9BY9CaywrKW8l3keqe
OjCLklb_2Vy9VcKny18Koc7glyO_54Eo4qAI7h6cIvM_ectzZc='
```

Decrypted Data:
My password is Python123

EXPERIMENT NUMBER :2

Upload and Analyze the data set given in csv format and perform data preprocessing and visualization.

CODE:

```
import pandas as pd
import matplotlib.pyplot as plt

# Create a dataset
data = {
   "Name": ["Arun", "Priya", "Ravi", "Meena"],
   "Marks": [80, 90, None, 70]
}

df = pd.DataFrame(data)

print("Original Data:")
print(df)

# Fill the missing mark with 0
df["Marks"] = df["Marks"].fillna(0)

print("Cleaned Data:")
print(df)

# Display a bar graph
plt.bar(df["Name"], df["Marks"])
plt.xlabel("Name")
plt.ylabel("Marks")
plt.title("Student Marks")
plt.show()
```

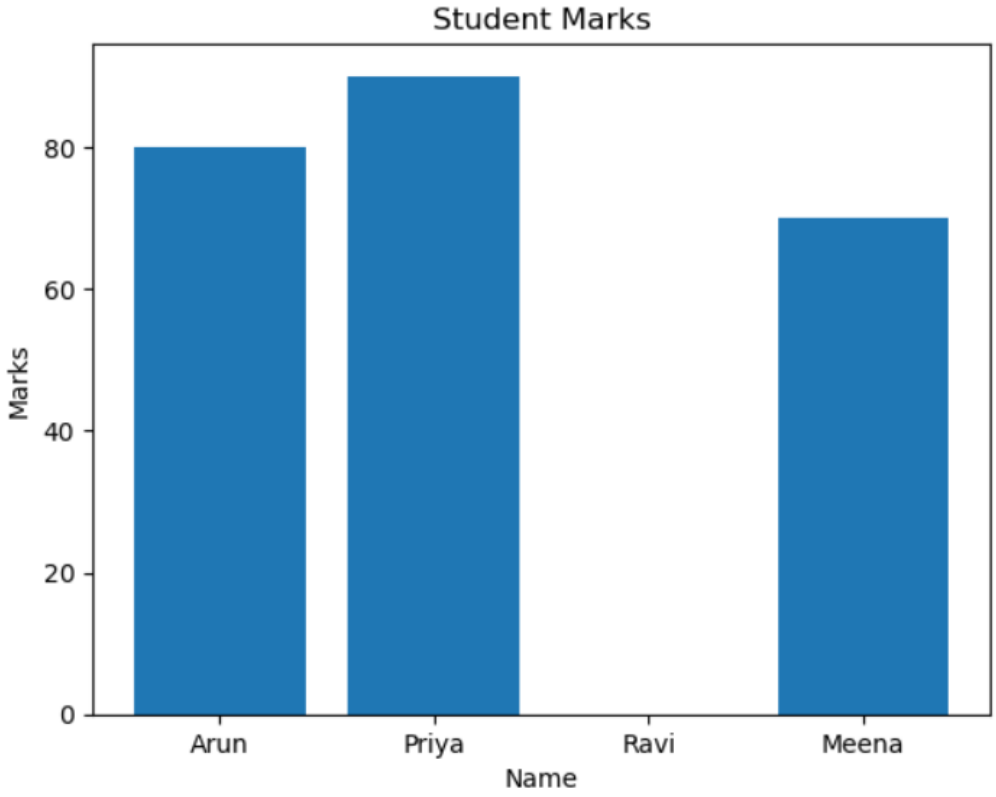
OUTPUT:

Original Data:

	Name	Marks
0	Arun	80.0
1	Priya	90.0
2	Ravi	NaN
3	Meena	70.0

Cleaned Data:

	Name	Marks
0	Arun	80.0
1	Priya	90.0
2	Ravi	0.0
3	Meena	70.0



EXPERIMENT NUMBER :3.a

Conduct an experiment to show data visualization using line plot.

CODE:

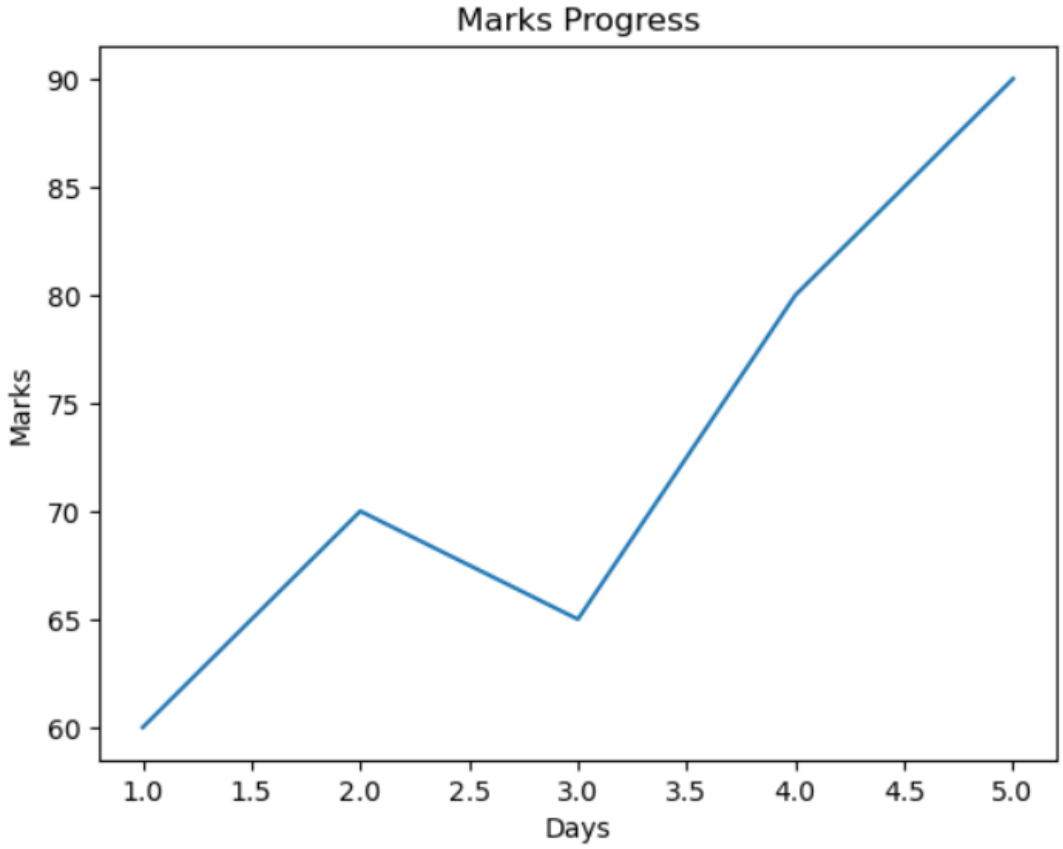
```
import matplotlib.pyplot as plt

days = [1, 2, 3, 4, 5]
```

```
marks = [60, 70, 65, 80, 90]

plt.plot(days, marks)
plt.xlabel("Days")
plt.ylabel("Marks")
plt.title("Marks Progress")
plt.show()
```

OUTPUT:



EXPERIMENT NUMBER :3.b

Conduct an experiment to show data visualization using bar chart.

CODE:

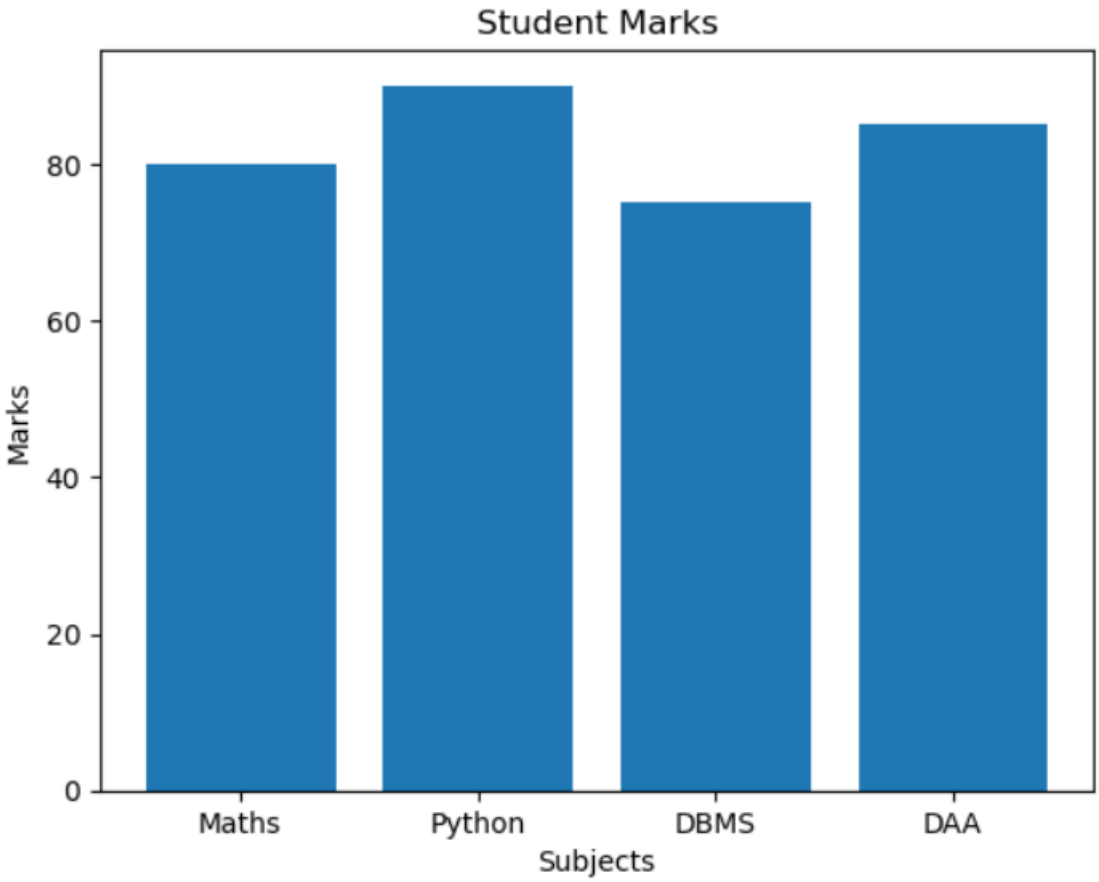
```
import matplotlib.pyplot as plt

subjects = ["Maths", "Python", "DBMS", "DAA"]
marks = [80, 90, 75, 85]

plt.bar(subjects, marks)
```

```
plt.xlabel("Subjects")
plt.ylabel("Marks")
plt.title("Student Marks")
plt.show()
```

OUTPUT:



EXPERIMENT NUMBER :3.c

Conduct an experiment to show data visualization using pie chart.

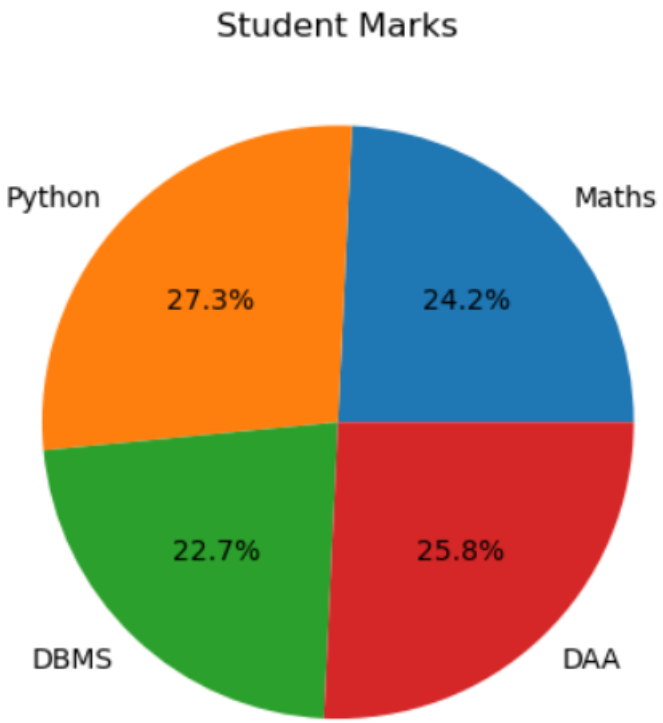
CODE:

```
import matplotlib.pyplot as plt

subjects = ["Maths", "Python", "DBMS", "DAA"]
marks = [80, 90, 75, 85]

plt.pie(marks, labels=subjects, autopct="%1.1f%%")
plt.title("Student Marks")
plt.show()
```

OUTPUT:



EXPERIMENT NUMBER :4

To Count the frequency of occurrence of a word in a body of text is often needed during text processing.

CODE:

```
text = input("Enter a text: ")
word = input("Enter a word: ")

count = text.lower().split().count(word.lower())

print("Frequency:", count)
```

OUTPUT:

Enter a text: python is easy and python is useful.
Enter a word: python
Frequency: 2

EXPERIMENT NUMBER :5

Data Collection and Initial Exploration.

CODE:

```
import pandas as pd

data = {
   "Name": ["Arun", "Priya", "Ravi", "Meena"],
   "Marks": [80, 90, 75, 85]
}

df = pd.DataFrame(data)

print("Collected Data:")
print(df)

print("\nFirst Two Rows:")
print(df.head(2))

print("\nAverage Marks:", df["Marks"].mean())
```

OUTPUT:

```
Collected Data:
   Name  Marks
0  Arun    80
1  Priya   90
2  Ravi    75
3  Meena   85

First Two Rows:
   Name  Marks
0  Arun    80
1  Priya   90

Average Marks: 82.5
```

EXPERIMENT NUMBER: 6

Time Series Analysis of Website Traffic

CODE:

```
import pandas as pd
```

```
import matplotlib.pyplot as plt

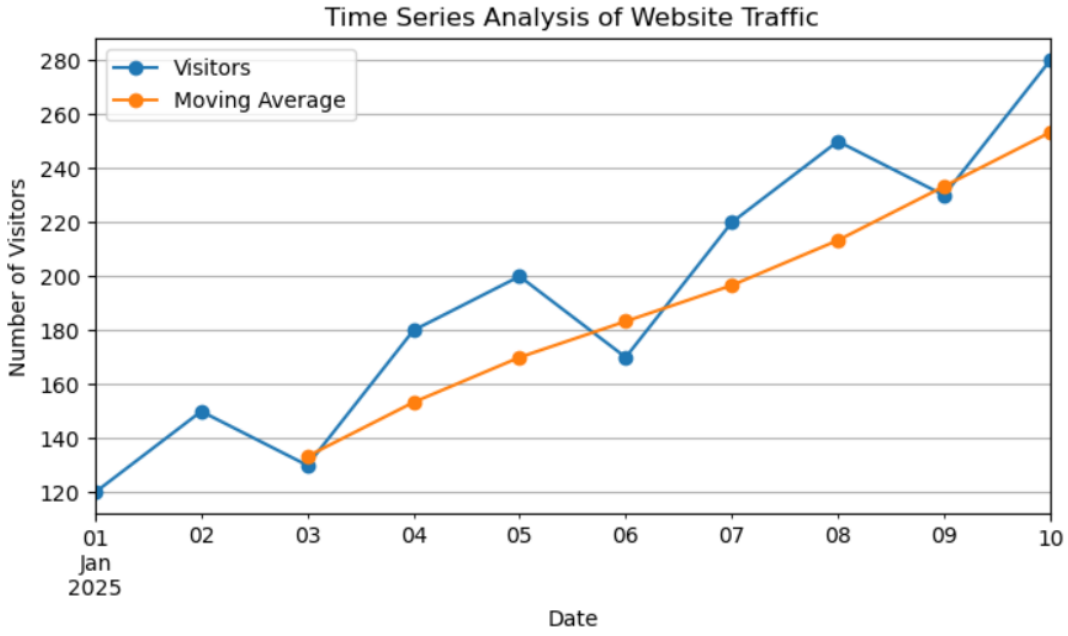
# Create sample website traffic data
data = {
   'Date': pd.date_range(start='2025-01-01', periods=10),
   'Visitors': [120, 150, 130, 180, 200, 170, 220, 250, 230, 280]
}

df = pd.DataFrame(data)
df.set_index('Date', inplace=True)

# Calculate moving average
df['Moving Average'] = df['Visitors'].rolling(window=3).mean()

# Plot time series
df.plot(figsize=(8, 4), marker='o')
plt.title('Time Series Analysis of Website Traffic')
plt.xlabel('Date')
plt.ylabel('Number of Visitors')
plt.grid()
plt.show()
```

OUTPUT:



EXPERIMENT NUMBER :7

Random Sampling and Sampling Distribution.

CODE:

```
import random
import matplotlib.pyplot as plt

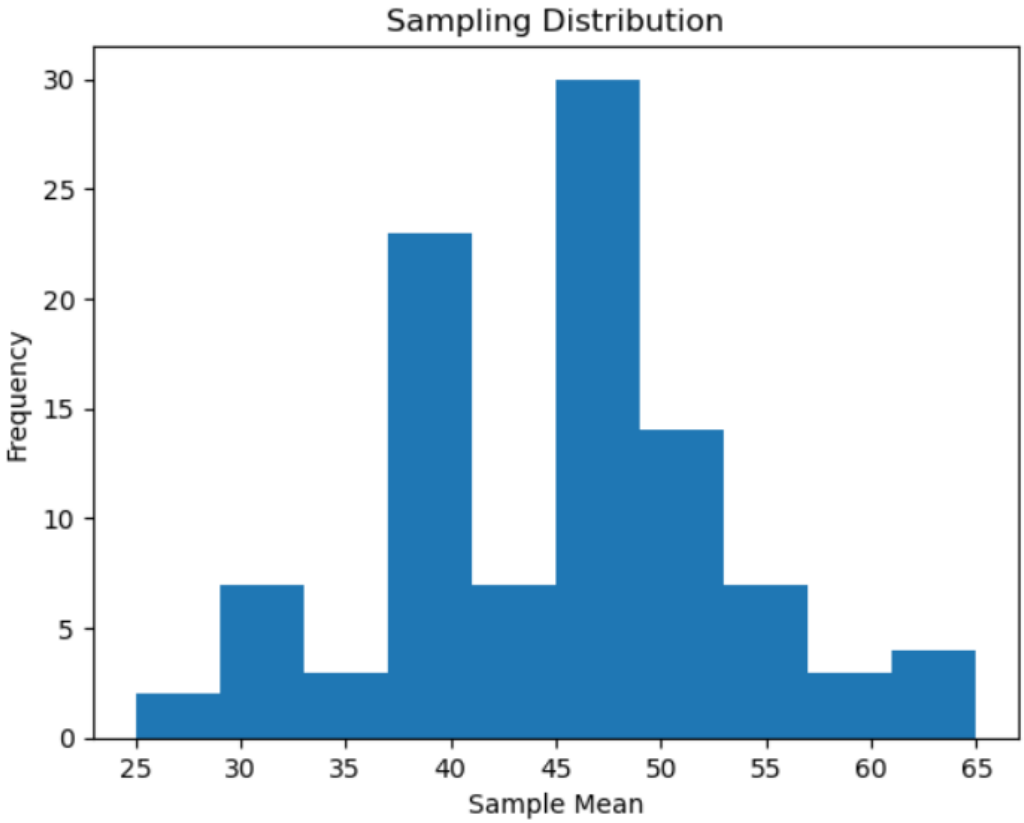
data = [10, 20, 30, 40, 50, 60, 70, 80]

means = []

for i in range(100):
    sample = random.sample(data, 4)
    means.append(sum(sample) / 4)

plt.hist(means, bins=10)
plt.xlabel("Sample Mean")
plt.ylabel("Frequency")
plt.title("Sampling Distribution")
plt.show()
```

OUTPUT:



EXPERIMENT NUMBER :8

Conduct Z test for the Data given using Python.

CODE:

```
import math

mean = float(input("Enter sample mean: "))
mu = float(input("Enter population mean: "))
sigma = float(input("Enter population standard deviation: "))
n = int(input("Enter sample size: "))

z = (mean - mu) / (sigma / math.sqrt(n))

print("Z-value:", round(z, 2))

if abs(z) > 1.96:
   print("Reject the null hypothesis")
else:
   print("Do not reject the null hypothesis")
```

OUTPUT:

```
Enter sample mean:  492
Enter population mean:  500
Enter population standard deviation:  20
Enter sample size:  40
Z-value: -2.53
Reject the null hypothesis
```

EXPERIMENT NUMBER :9

Experiment to understand Matplotlib library use cases in Data Science.

CODE:

```
import matplotlib.pyplot as plt
```

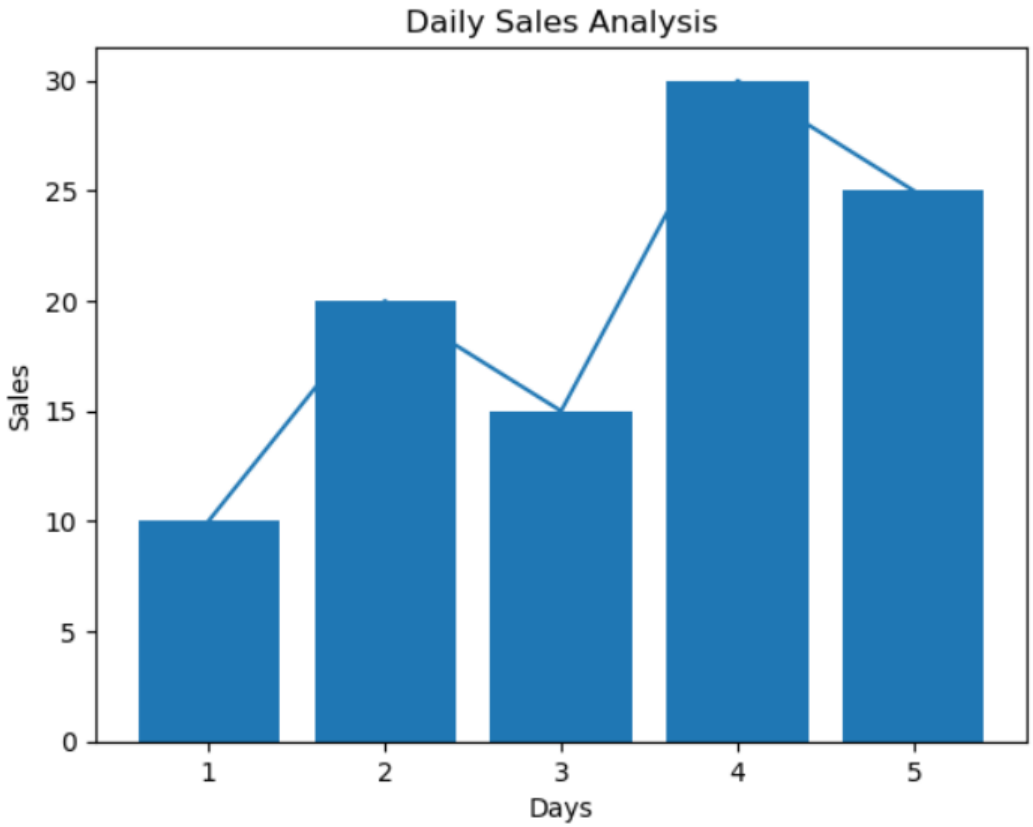
```
days = [1, 2, 3, 4, 5]
sales = [10, 20, 15, 30, 25]

plt.plot(days, sales, label="Sales")
plt.bar(days, sales)

plt.xlabel("Days")
plt.ylabel("Sales")
plt.title("Daily Sales Analysis")

plt.show()
```

OUTPUT:



EXPERIMENT NUMBER :10

Experiment to understand array function in Data science.

CODE:

```
import numpy as np
```

```
a = np.array([10, 20, 30, 40, 50])

print("Array:", a)
print("Sum:", np.sum(a))
print("Mean:", np.mean(a))
print("Maximum:", np.max(a))
print("Minimum:", np.min(a))
print("Sorted Array:", np.sort(a))
```

OUTPUT:

```
Array: [10 20 30 40 50]
Sum: 150
Mean: 30.0
Maximum: 50
Minimum: 10
Sorted Array: [10 20 30 40 50]
```

EXPERIMENT NUMBER :11

Experiment to understand pandas library use cases Data science.

CODE:

```
import pandas as pd

data = {
   "Name": ["Arun", "Priya", "Ravi", "Meena"],
   "Marks": [80, 90, 75, 85]
}

df = pd.DataFrame(data)

print("Student Data:")
print(df)

print("Average Marks:", df["Marks"].mean())
print("Highest Marks:", df["Marks"].max())
```

```
print("Lowest Marks:", df["Marks"].min())
```

OUTPUT:

```
Student Data:
   Name  Marks
0   Arun    80
1  Priya    90
2   Ravi    75
3  Meena    85
Average Marks: 82.5
Highest Marks: 90
Lowest Marks: 75
```

EXPERIMENT NUMBER :12

Experiment to detect outliers in a given data set.

CODE:

```
data = [10, 12, 14, 15, 18, 20, 100]


q1 = 12
q3 = 20
iqr = q3 - q1

for x in data:
    if x < q1 - 1.5 * iqr or x > q3 + 1.5 * iqr:
        print("Outlier:", x)
```

OUTPUT:

```
Outlier: 100
```

EXPERIMENT NUMBER :13

Experiment to handle missing values and inappropriate data in a given data set.

CODE:

```
import pandas as pd


data = {
   "Name": ["Arun", "Priya", "Ravi", "Meena"],
   "Marks": [80, None, 75, 150]
}

df = pd.DataFrame(data)

print("Original Data:")
print(df)

df["Marks"] = df["Marks"].fillna(0)
df.loc[df["Marks"] > 100, "Marks"] = 0

print("\nCleaned Data:")
print(df)
```

OUTPUT:

```
Original Data:
   Name  Marks
0  Arun   80.0
1  Priya   NaN
2  Ravi   75.0
3  Meena  150.0

Cleaned Data:
   Name  Marks
0  Arun   80.0
1  Priya   0.0
2  Ravi   75.0
```

3 Meena 0.0

EXPERIMENT NUMBER :14

Experiment to understand feature scaling.

CODE:

```
data = [10, 20, 30, 40, 50]

minimum = min(data)
maximum = max(data)

for x in data:
   scaled = (x - minimum) / (maximum - minimum)
   print(round(scaled, 2))
```

OUTPUT:

0.0

0.25

0.5

0.75

1.0

EXPERIMENT NUMBER :15

Experiment to understand the data preprocessing in Data science.

CODE:

```
import pandas as pd


data = {
   "Name": ["Arun", "Priya", "Ravi", "Meena"],
   "Marks": [80, None, 75, 150]
}

df = pd.DataFrame(data)
```

```
print("Original Data:")
print(df)

df["Marks"] = df["Marks"].fillna(0)
df.loc[df["Marks"] > 100, "Marks"] = 100

print("\nPreprocessed Data:")
print(df)
```

OUTPUT:

```
Original Data:
   Name  Marks
0  Arun   80.0
1  Priya   NaN
2  Ravi   75.0
3  Meena  150.0

Preprocessed Data:
   Name  Marks
0  Arun   80.0
1  Priya   0.0
2  Ravi   75.0
3  Meena  100.0
```

EXPERIMENT NUMBER :16

Experiment to understand EDA-Quantitative and Qualitative analysis.

CODE:

```
import pandas as pd

data = {
   "Name": ["Arun", "Priya", "Ravi", "Meena"],
   "Marks": [80, 90, 75, 85]
}
```

```
df = pd.DataFrame(data)

print("Dataset:")
print(df)

print("\nQuantitative Analysis:")
print("Average Marks:", df["Marks"].mean())
print("Maximum Marks:", df["Marks"].max())
print("Minimum Marks:", df["Marks"].min())

print("\nQualitative Analysis:")
print(df["Name"])
```

OUTPUT:

```
Dataset:
   Name  Marks
0  Arun    80
1  Priya   90
2  Ravi    75
3  Meena   85

Quantitative Analysis:
Average Marks: 82.5
Maximum Marks: 90
Minimum Marks: 75

Qualitative Analysis:
0    Arun
1   Priya
2    Ravi
3   Meena
Name: Name, dtype: str
```

EXPERIMENT NUMBER :17

Experiment to understand Linear regression for a given data set.

CODE:

```
import matplotlib.pyplot as plt

from sklearn.linear_model import LinearRegression


# Given dataset
X = [[1], [2], [3], [4], [5]]
Y = [2, 4, 5, 4, 5]

# Create the model
model = LinearRegression()

# Train the model
model.fit(X, Y)

# Predict the values
Y_pred = model.predict(X)

# Display the results
print("Slope:", model.coef_[0])
print("Intercept:", model.intercept_)
print("Predicted values:", Y_pred)

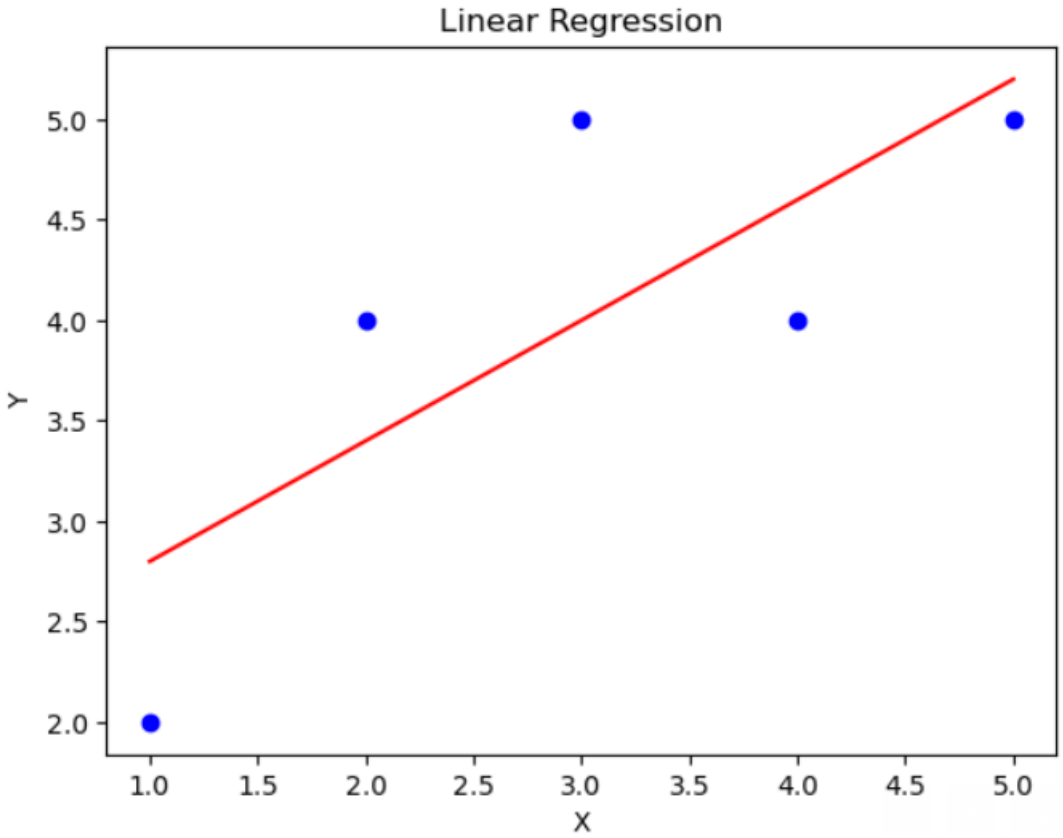
# Draw the graph
plt.scatter(X, Y, color="blue")
plt.plot(X, Y_pred, color="red")
plt.xlabel("X")
plt.ylabel("Y")
plt.title("Linear Regression")
plt.show()
```

OUTPUT:

```
Slope: 0.6
```

Intercept: 2.2

Predicted values: [2.8 3.4 4. 4.6 5.2]



EXPERIMENT NUMBER :18

Experiment to understand KNN algorithm for a given data set

CODE:

```
from sklearn.neighbors import KNeighborsClassifier


# Given dataset
X = [[1], [2], [3], [6], [7], [8]]
Y = ['A', 'A', 'A', 'B', 'B', 'B']

# Create the KNN model
model = KNeighborsClassifier(n_neighbors=3)

# Train the model
model.fit(X, Y)
```

```
# Predict the class of a new value
new_value = [[5]]
prediction = model.predict(new_value)

# Display the result
print("Input value:", new_value[0][0])
print("Predicted class:", prediction[0])
```

OUTPUT:

Input value: 5

Predicted class: B

EXPERIMENT NUMBER :19

Experiment to understand Logistic Regression for a given data set.

CODE:

```
from sklearn.linear_model import LogisticRegression


# Given dataset
X = [[1], [2], [3], [4], [5], [6]]
Y = [0, 0, 0, 1, 1, 1]

# Create the Logistic Regression model
model = LogisticRegression()

# Train the model
model.fit(X, Y)

# Predict the class of a new value
new_value = [[4]]
prediction = model.predict(new_value)

# Display the result
print("Input value:", new_value[0][0])
```

```
print("Predicted class:", prediction[0])
```

OUTPUT:

Input value: 4

Predicted class: 1

EXPERIMENT NUMBER :20

Experiment to understand K-means clustering algorithm for a given data set.

CODE:

```
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
import warnings

# Ignore the environment warning
warnings.filterwarnings("ignore", category=RuntimeWarning)

# Given dataset
X = [[1, 2], [2, 3], [3, 4],
    [8, 9], [9, 10], [10, 11]]

# Create the K-Means model
model = KMeans(n_clusters=2, random_state=0, n_init=10)

# Train the model
model.fit(X)

# Display cluster labels
print("Cluster Labels:", model.labels_)

# Plot the clusters
plt.scatter(
   [1, 2, 3, 8, 9, 10],
```

```
    [2, 3, 4, 9, 10, 11],
    c=model.labels_,
    s=100
)

# Add title and labels
plt.title("K-Means Clustering")
plt.xlabel("X")
plt.ylabel("Y")

# Display the graph
plt.show()
```

OUTPUT:

Cluster Labels: [1 1 1 0 0 0]

